

```
In [1]: import json
from pathlib import Path
import shutil

WEB_DIR = Path("phase1_web_nn")
WEB_DIR.mkdir(parents=True, exist_ok=True)
MODEL_ONNX = Path("best_model.onnx")
FEATURES_JSON = WEB_DIR / "features.json"

# Update this to match the input size used in ml.ipynb
INPUT_SIZE = 200

# Optional: provide friendly feature names if you have them
FEATURE_NAMES = [f"x{i}" for i in range(INPUT_SIZE)]

if not MODEL_ONNX.exists():
    raise FileNotFoundError("best_model.onnx not found. Export it from notebooks/dsai/phase2_mlflow_nn.ipynb.")

shutil.copy2(MODEL_ONNX, WEB_DIR / "best_model.onnx")

with FEATURES_JSON.open("w", encoding="utf-8") as f:
    json.dump({"input_size": INPUT_SIZE, "feature_names": FEATURE_NAMES}, f, indent=2)

index_html = """<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>NN Predictor</title>
  <link rel="stylesheet" href="style.css" />
  <script src="https://cdn.jsdelivr.net/npm/onnxruntime-web@1.19.0/dist/ort.min.js"></script>
</head>
<body>
  <main class="container">
    <header>
      <h1>NN Predictor</h1>
      <p>Neural network model exported from PyTorch (best_model.onnx).</p>
    </header>
    <section id="form"></section>
    <section class="output">
      <div>Prediction:</div>
      <pre id="pred" class="pred">[]</pre>
    </section>
  </main>
  <script src="app.js"></script>
</body>
</html>
"""

style_css = """* { box-sizing: border-box; }
body {
  font-family: "Georgia", "Times New Roman", serif;
  margin: 0;
  background: radial-gradient(circle at 20% 20%, #f6efe6, #e4f0e2);
  color: #1f2a2d;
}
.container {
  max-width: 980px;
  margin: 0 auto;
  padding: 32px 24px 60px;
}
#form {
  margin: 22px 0 28px;
}
table {
  width: 100%;
  border-collapse: collapse;
  background: #ffffffcc;
  border: 1px solid #d9d2c4;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0 4px 12px rgba(0,0,0,0.05);
}
th, td {
  border-bottom: 1px solid #e1d9c9;
  padding: 8px 10px;
}
th {
  text-align: left;
  background: #efe6da;
}
td input {
  width: 100%;
}
thead th:first-child, tbody td:first-child {
  width: 12%;
}
thead th:not(:first-child), tbody td:not(:first-child) {
  width: 22%;
}
```



```

}
@media (max-width: 720px) {
  thead {
    display: none;
  }
  table, tbody, tr, td {
    display: block;
    width: 100%;
  }
  tr {
    margin-bottom: 12px;
    border-bottom: 1px solid #d9d2c4;
  }
  td {
    border: none;
  }
  td::before {
    content: attr(data-label);
    font-weight: 600;
    display: block;
    margin-bottom: 4px;
  }
}
.output {
  background: #1f2a2d;
  color: #f8f4ee;
  padding: 18px 20px;
  border-radius: 12px;
}
.pred { font-size: 16px; margin: 6px 0 0; }
"""

app_js = """const formEl = document.getElementById('form');
const predEl = document.getElementById('pred');
let session = null;
let inputSize = 0;

async function loadModel() {
  session = await ort.InferenceSession.create('best_model.onnx');
  const r = await fetch('features.json');
  const data = await r.json();
  inputSize = data.input_size;
  buildTable(inputSize);
  await updatePrediction();
}

function buildTable(size) {
  const rows = Math.ceil(size / 4);
  const table = document.createElement('table');
  const thead = document.createElement('thead');
  const headRow = document.createElement('tr');
  ['index', 'time', 'x', 'y', 'z'].forEach((name) => {
    const th = document.createElement('th');
    th.textContent = name;
    headRow.appendChild(th);
  });
  thead.appendChild(headRow);
  table.appendChild(thead);
  const tbody = document.createElement('tbody');
  for (let i = 0; i < rows; i += 1) {
    const row = document.createElement('tr');
    const indexCell = document.createElement('td');
    indexCell.setAttribute('data-label', 'index');
    indexCell.textContent = String(i);
    row.appendChild(indexCell);
    ['time', 'x', 'y', 'z'].forEach((label, colIndex) => {
      const td = document.createElement('td');
      td.setAttribute('data-label', label);
      const input = document.createElement('input');
      input.type = 'number';
      input.step = '0.01';
      input.value = '0';
      input.dataset.index = i * 4 + colIndex;
      if (Number(input.dataset.index) >= size) {
        input.disabled = true;
      }
      input.addEventListener('input', () => updatePrediction());
      td.appendChild(input);
      row.appendChild(td);
    });
    tbody.appendChild(row);
  }
  table.appendChild(tbody);
  formEl.innerHTML = '';
  formEl.appendChild(table);
}

async function updatePrediction() {
  if (!session) return;
  const inputs = Array.from(formEl.querySelectorAll('input'))

```



```

        .filter((el) => !el.disabled)
        .map((el) => Number(el.value));
const inputTensor = new ort.Tensor('float32', Float32Array.from(inputs), [1, inputs.length]);
const results = await session.run({ input: inputTensor });
const outputName = Object.keys(results)[0];
predEl.textContent = JSON.stringify(Array.from(results[outputName].data));
}

loadModel();
"""

(WEB_DIR / "index.html").write_text(index_html, encoding="utf-8")
(WEB_DIR / "style.css").write_text(style_css, encoding="utf-8")
(WEB_DIR / "app.js").write_text(app_js, encoding="utf-8")

print(f"Wrote static app to {WEB_DIR}")
```

Wrote static app to phase1_web_nn

In []: !cd phase1_web_nn && python3 -m http.server 8000

```
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [20/Apr/2026 10:54:01] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 10:54:01] "GET /style.css HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 10:54:01] "GET /app.js HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 10:54:01] code 404, message File not found
127.0.0.1 - - [20/Apr/2026 10:54:01] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [20/Apr/2026 10:54:03] "GET /best_model.onnx HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 10:54:03] "GET /features.json HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 10:54:20] "GET / HTTP/1.1" 304 -
127.0.0.1 - - [20/Apr/2026 10:54:20] "GET /style.css HTTP/1.1" 304 -
127.0.0.1 - - [20/Apr/2026 10:54:20] "GET /app.js HTTP/1.1" 304 -
127.0.0.1 - - [20/Apr/2026 10:54:20] "GET /features.json HTTP/1.1" 304 -
```